

introduzione a java

flavio

May 11, 2026

Contents

1	Tipi di dato base	1
1.1	Un tipo di dato descrive un insieme di valori e di operazioni definite su quei valori:	1
1.2	Tutti i tipi built-in	2
1.3	Precedenza operatori aritmetici	2
2	Variabili, dichiarazioni e assegnazioni	2
3	Letterali	3
3.1	per distinguere gli interi da i numeri in virgola mobile	3
3.2	Virgola mobile	3
4	Espressioni	3
5	Caratteri e Stringhe	3
5.1	Caratteri di escape	3
5.1.1	breve lista	4
6	Priorità operatori	4
7	Conversioni di tipo	4
7.1	conversione automatica a stringa	4
7.2	Conversione esplicita	5
7.3	Cast esplicito	5
7.4	Cast implicito	5
7.4.1	Avviene o in fase di assegnazione: byte, short e char possono essere promossi a int.	5
7.4.2	calcolo expression	5

8 Istruzioni di controllo	5
8.1 if ed else	6
8.2 Switch	6
9 Istruzioni iterative	7
9.1 while	7
9.2 break e i cicli annidati	7

1 Tipi di dato base

I tipi di base sono built-in, non sono creati dall'utente. Serve esserne a conoscenza per ragioni di efficienza.

1.1 Un tipo di dato descrive un insieme di valori e di operazioni definite su quei valori:

Tipo	Dominio	Operatori
int	Interi	+ - * / %
double	Numeri in virgola mobile	
boolean	valori booleani	&&
char	caratteri	+ -

1.2 Tutti i tipi built-in

Tipo	Intervallo	Dimensione
float	parte intera: ± 1038 , parte frazionaria: circa 7 cifre decimali significative	4 byte
double	parte intera ± 10308 , parte frazionaria: circa 15 cifre decimali significative	8 byte
int	-2147483648...2147483647	4 byte
long	-2147483648...2147483647	4 byte
boolean	true,false	1 byte
char	rappresenta tutti i caratteri codificati con unicode	2 byte
short	-32768...32767	2 byte
byte	-128...127	1 byte

1.3 Precedenza operatori aritmetici

Operatori	Operazioni	Precedenza
*,/,%	Moltiplicazione,divisione,resto	Valutati per primi, da sinistra verso destra
+, -	Addizione,Sottrazione	Valutati per secondi, da sinistra verso destra

2 Variabili, dichiarazioni e assegnazioni

Una variabile è un nome usato per riferirsi a un valore di un tipo di dato

- Può essere dichiarata (`int a;`) e quindi non è detto che abbia un valore, se non in certi contesti specifici.
 - Il valore viene assegnato a una variabile mediante assegnazione `a = 0;`
 - * Si può anche dichiarare e assegnare allo stesso tempo: `int a = 0;`

```
int a,b; // dichiarazione
a = 5; // assegnazione
b = a+10; // altra assegnazione
int c = a+b; // dichiarazione e assegnazione
a = c-3; // assegnazione
```

3 Letterali

Un letterale è una rappresentazione nel codice sorgente del valore di un tipo di dato.

- 27 e 32 sono letterali per gli interi
 - 3.14 è un letterale per i double
 - * `true` o `false` sono gli unici due letterali per il tipo booleano
 - `"ciao mondo"` è un letterale per il tipo String

3.1 per distinguere gli interi da i numeri in virgola mobile

- Le costanti di tipo `int` sono semplicemente numeri nell'intervallo `[-2000000000,+2000000000]`
 - Le costanti di tipo `long` sono specificate con il suffisso `l` o `L`

3.2 Virgola mobile

- `double`: numeri con la virgola
 - `float`: hanno il suffisso `f` o `F`: `10.5f`

4 Espressioni

Un' Espressione è un letterale, una variabile o una sequenza di operazioni su letterali o varibili che producono un valore.

```
int a = 42*(x-2)+y;
```

5 Caratteri e Stringhe

Un `char` è un carattere alfanumerico o un simbolo. Ci sono 2^{16} possibili valori di caratteri.

5.1 Caratteri di escape

In alcuni casi dobbiamo rappresentare caratteri speciali o non stampabili.

5.1.1 breve lista

- Tab: `'\t'`
- A capo: `'\n'`
- Backlash: `'\\'`
- Apice: `'\"'`
- Virgolette: `'\"'`

6 Priorità operatori

Operatori	Operazioni
Post-incremento e post-decremento	<code>++var --var +espr -espr</code>
Operatori moltiplicativi	<code>* / %</code>
Operatori additivi	<code>+ -</code>
shift	<code><< >> >>></code>
relazionali	<code>< > <= >= instanceof</code>
confronto	<code>~ = ! ~</code>
And bit a bit	<code>&</code>
XOR	<code>^</code>
OR	
OR logico	
operatore ternario	<code>?:</code>
lambda	<code>-></code>
assegnazione	<code>= += -= *= /= %= &= ^= <= >= >>=</code>

7 Conversioni di tipo

Supponiamo di voler utilizzare il programma per effettuare dei calcoli. Come convertiamo da `stringa` a `intero`?

7.1 conversione automatica a stringa

Quando cerchiamo di concatenare a una stringa un operando di tipo diverso viene convertito automaticamente a stringa.

```
public class PensieroProfondo
{
    public static void main(String[] args)
    {
        String s = "La risposta alla fondamentale sulla vita, l'universo e tutto";
        int v = 42;
        String risposta = s+v; // equivalente a String risposta = s+Integer.pa
        System.out.println(risposta);
    }
}
```

7.2 Conversione esplicita

Utilizzando un metodo che prende in ingresso un argomento di un tipo e restituisce un valore di un altro tipo. `Integer.parseInt(int n)`, `Double.parseDouble(double`

```
n), Math.round(double n), Math.floor(double n), Math.ceil(double n)
```

7.3 Cast esplicito

Anteponendo il tipo desiderato tra parentesi all'espressione essa verrà convertita.

```
System.out.println((int)2.781828);
```

7.4 Cast implicito

7.4.1 Avviene o in fase di assegnazione: byte, short e char possono essere promossi a int.

-int può essere promosso a long -float a double

7.4.2 calcolo espressione

se uno dei due operandi è double, l'intera espressione è promossa a double. Altrimenti se uno dei due operandi è float, l'intera espressione è promossa a float

8 Istruzioni di controllo

Finora abbiamo controllato il flusso di esecuzione attraverso la sequenza (l'ordine di esecuzione delle istruzioni). Mediante le istruzioni di controllo condizionali possiamo controllare il flusso del programma.

8.1 if ed else

Per realizzare una decisione si usa l'istruzione if.

- La sintassi è `if(<espressione booleana>)` solo un'istruzione oppure: `if (<espressione booleana>) { }`

```
if (x < 0) x = -x; // calcola valore assoluto
if(x < y){ //credit default swap
    int t = x;
    x = y;
    y = t;
}
```

- l'**else che è facoltativo** viene eseguito se la condizione è falsa. Bisogna fare attenzione quando si annidano i diversi if ed else: l'else si riferisce sempre all'ultimo .

```

if(x > 0)
    if(y > 0)
        System.out.println("x e y sono > 0");
else
    System.out.println("x <= 0")
if(x > 0)
    if(y > 0)
        if(z < 5)
            if(w >= 3)
                System.out.println("x e y >= 0 e z <= 5 e w >= 3")

```

8.2 Switch

Per confrontare una espressione intera si può usare l'espressione `switch`

```

String hello = null;
switch(new Random.nextInt(6))
{
    case 0: hello = "Ciao" break;
    case 1: hello = "hello" break;
    // etc ...
}

//versione più compatta
String s = switch(c) // converte un carattere in rappresentazione
{
    switch 'k' -> "kappa";
    switch 'h' -> "acca";
    switch 'l' -> "elle";
    default -> "non so leggerlo";
}

```

9 Istruzioni iterative

Ci sono 3 tipi di istruzioni iterative: `while`, `for`, `do while`

9.1 while

sintassi: `while(<espressione>) { fino a quando espressione è vera }` semantica: eseguito fino

9.2 break e i cicli annidati

Il `break` ci permette di uscire solamente dal ciclo corrente se non si specifica una etichetta.

```
outer:
for (int i = 0; i < h; i++)
{
    for (int j = 0; j < w; j++)
    {
        if (i == j) break outer;
    }
}
```